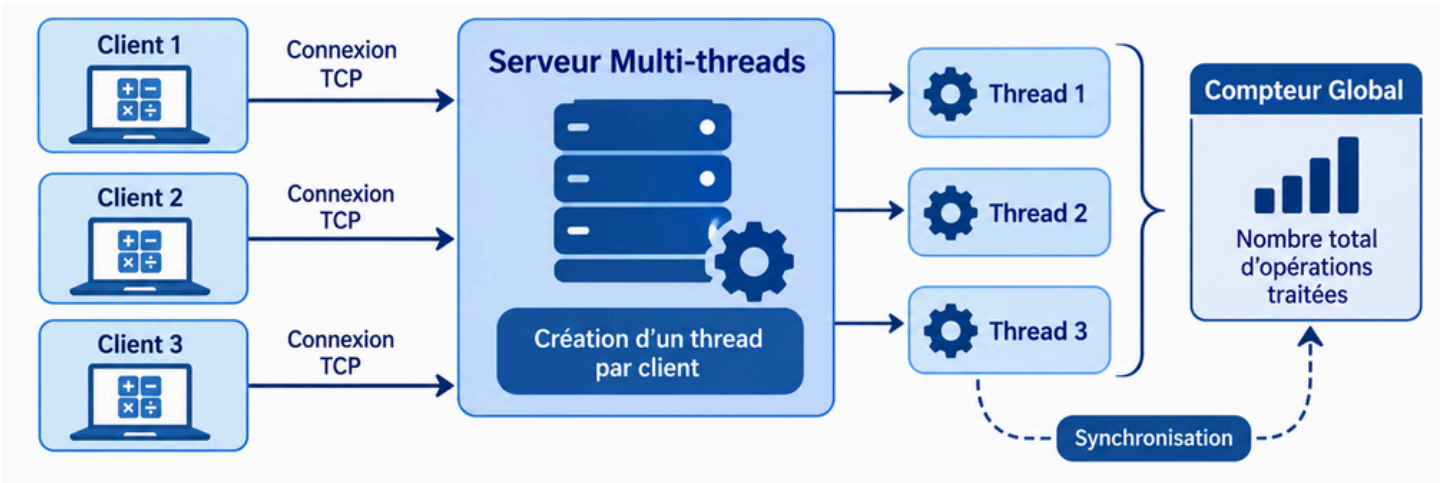
COMPTE RENDU TP3 A3.2

EYA CHIKHAOUI

3e Licence Sciences de l'Informatique (LSI 3 – 1.2)

1. Introduction :

- Dans cette deuxième activité du TP3, nous avons pour objectif d'améliorer l'application calculatrice développée précédemment en utilisant la communication client-serveur par échange d'objets Java.
- Cette activité consiste à reprendre le mécanisme de sérialisation des objets étudié dans le TP2 et à l'intégrer dans un serveur multithread. Chaque client pourra envoyer un objet représentant une opération mathématique. Le serveur recevra cet objet, effectuera le calcul correspondant et transmettra le résultat au client.
- Nous devons également mettre en place un compteur global partagé entre les différents threads du serveur. L'accès à ce compteur devra être synchronisé afin d'éviter les conflits lorsque plusieurs clients effectuent des opérations simultanément.



2. Objectifs de l'activité

Cette activité a pour objectif principal d'améliorer l'application calculatrice développée précédemment en intégrant la communication par échange d'objets Java dans un serveur multithread.

Les objectifs spécifiques sont les suivants :

- Réutiliser le mécanisme de sérialisation des objets Java à l'aide de l'interface Serializable.
- Permettre au client d'envoyer un objet contenant les informations d'une opération mathématique au serveur.
- Adapter le serveur afin de traiter plusieurs clients simultanément grâce à la création de threads.
- Effectuer les opérations mathématiques demandées et transmettre les résultats aux clients.
- Mettre en place un compteur global partagé entre les différents threads.
- Utiliser la synchronisation afin de garantir un accès contrôlé au compteur lors des traitements simultanés.

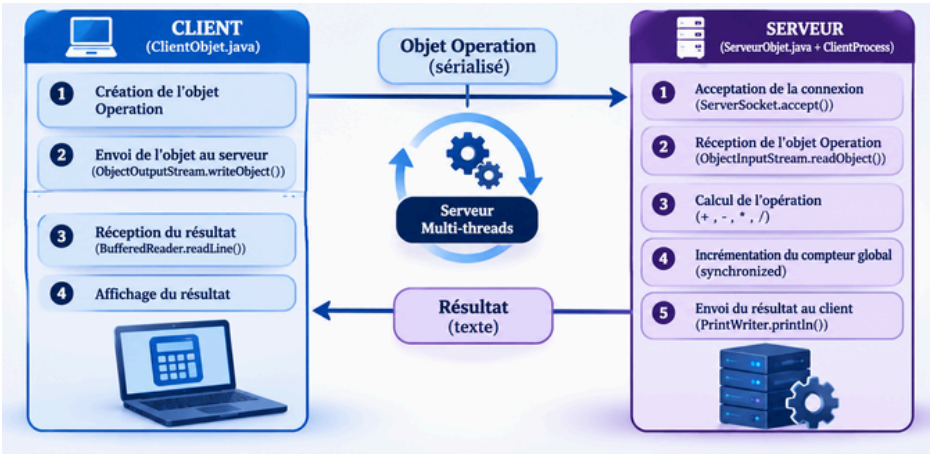


figure 1 : principe général

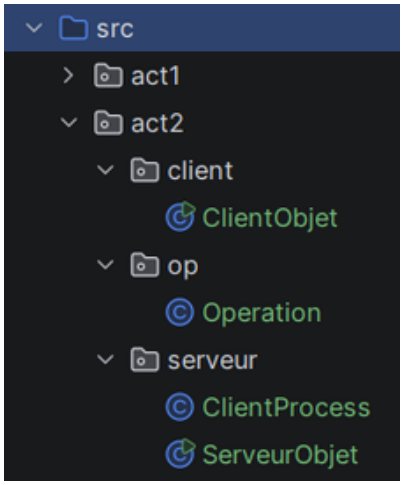


figure 2 : Arborescence du projet

3. Organisation du projet

Pour réaliser cette activité, nous avons créé un nouveau package act2 dans le projet Java TP3-DAR2026.
Ce package est organisé en trois sous-packages :

- client :
  - contient le programme client.
- op :
  - contient la classe Operation, utilisée pour représenter les opérations mathématiques.
- serveur :
  - contient la classe ServeurObjet, qui assure l'écoute et l'acceptation des connexions TCP, ainsi que la classe ClientProcess, chargée du traitement indépendant des opérations de chaque client dans un thread.

4. Création de la classe Operation

La première étape consiste à créer la classe Operation dans le package act2.op.
Cette classe implémente l'interface Serializable, ce qui permet de transmettre ses objets à travers une connexion réseau.

Elle contient trois attributs :

- operande1 : le premier nombre de l'opération.
- operande2 : le deuxième nombre de l'opération.
- operateur : l'opérateur mathématique à appliquer.

Un constructeur permet d'initialiser ces attributs, tandis que les méthodes getters permettent au serveur d'accéder aux valeurs reçues.

Operation.java

```java
package act2.op;
import java.io.Serializable;

public class Operation implements Serializable {  no usages  new *

    private static final long serialVersionUID = 1L;  no usages

    private double operande1;  2 usages
    private double operande2;  2 usages
    private String operateur;  2 usages

    public Operation(double operande1, String operateur, double operande2) {
        this.operande1 = operande1;
        this.operateur = operateur;
        this.operande2 = operande2;
    }

    public double getOperande1() {  no usages  new *
        return operande1;
    }

    public double getOperande2() {  no usages  new *
        return operande2;
    }

    public String getOperateur() {  no usages  new *
        return operateur;
    }
}
```

Figure 3 –Code source de la classe Operation

5. Création de la classe ClientProcess

Nous avons créé la classe ClientProcess dans le package act2.serveur.
Cette classe implémente l'interface Runnable afin de permettre le traitement indépendant de chaque client dans un thread.

Lorsqu'un client se connecte, le serveur crée un objet ClientProcess associé à son socket. Le thread correspondant reçoit ensuite l'objet Operation transmis par le client, récupère les données nécessaires et effectue le calcul demandé.

Après le calcul, le thread demande l'incrémentation du compteur global, puis transmet le résultat au client à travers un flux de sortie.

ClientProcess.java

```java
package act2.serveur;
import java.io.ObjectInputStream;
import java.io.PrintWriter;
import java.io.IOException;
import java.net.Socket;
import act2.op.Operation;
public class ClientProcess implements Runnable {  1 usage  new *
    private Socket socketClient;  4 usages
    public ClientProcess(Socket socketClient) {  1 usage  new *
        this.socketClient = socketClient;
    }
    @Override  new *
    public void run() {
        try {
            // Creation des flux
            ObjectInputStream ois = new ObjectInputStream(socketClient.getInputStream());
            PrintWriter out = new PrintWriter(socketClient.getOutputStream(), autoFlush: true);
            // Reception de l'objet Operation
            Operation operation = (Operation) ois.readObject();
            System.out.println("Operation recue : "
                    + operation.getOperande1() + " " + operation.getOperateur() + " " + operation.getOperande2());
            // Calcul du resultat
            double resultat = calculer(operation);
            System.out.println("Resultat calcule : " + resultat);
            // Incrementation du compteur global
            int compteur = ServeurObjet.incrementerCompteur();
            System.out.println("Nombre total d'operations : " + compteur);
            // Envoi du resultat au client
            out.println(resultat);
        } catch (IOException | ClassNotFoundException e) {
            System.out.println("Erreur lors du traitement du client : " + e.getMessage());
        } finally {
            try {
                socketClient.close();
            } catch (IOException e) {
                System.out.println("Erreur lors de la fermeture du socket.");
            }
        }
    }
    private double calculer(Operation op) {  1 usage  new *
        double a = op.getOperande1();
        double b = op.getOperande2();
        String operateur = op.getOperateur();
        switch (operateur) {
            case "+":
                return a + b;
            case "-":
                return a - b;
            case "*":
                return a * b;
            case "/":
                return b != 0 ? a / b : Double.NaN;
            default:
                return Double.NaN;
        }
    }
}
```

Figure 4 – Code source de la classe ClientProcess

6. Création du serveur multithread

Nous avons ensuite créé la classe ServeurObjet, qui assure l'écoute des connexions sur le port 1234.
Le serveur utilise la classe ServerSocket pour accepter les connexions entrantes.
À chaque nouvelle connexion, il crée un objet ClientProcess, puis démarre un nouveau thread avec la méthode start().
Cette organisation permet au serveur de traiter plusieurs clients indépendamment, sans attendre la fin du traitement d'un client pour accepter les connexions suivantes.

ServeurObjet.java

```java
package act2.serveur;
import java.io.IOException;
import java.net.ServerSocket;
import java.net.Socket;

public class ServeurObjet {  new *
    private static final int PORT = 1234;  2 usages
    private static int compteurGlobal = 0;  2 usages

    // Methode synchronisee pour incrementer le compteur
    public static synchronized int incrementerCompteur() {  1 usage  new *
        compteurGlobal++;
        return compteurGlobal;
    }
    public static void main(String[] args) {  new *
        int numeroClient = 0;
        try (
                ServerSocket serveurSocket = new ServerSocket(PORT)) {
            System.out.println("Serveur demarre...");
            System.out.println("En attente des connexions sur le port " + PORT);
            while (true) {
                Socket socketClient = serveurSocket.accept();
                numeroClient++;
                System.out.println("Client n°" + numeroClient + " connecte depuis : " + socketClient.getRemoteSocketAddress());
                Thread thread = new Thread(new ClientProcess(socketClient)
                );
                thread.start();
            }
        } catch (IOException e) {
            System.out.println("Erreur du serveur : " + e.getMessage());
        }
    }
}
```

Figure 5 – Code source de la classe ServeurObjet

7. Mise en place du compteur global synchronisé

Conformément à l'énoncé, nous avons ajouté un compteur global partagé entre les différents traitements des clients.
La variable compteurGlobal est déclarée avec le mot-clé static, ce qui permet à tous les threads d'utiliser la même variable.
Pour éviter les conflits d'accès lorsque plusieurs clients effectuent des opérations simultanément, nous avons utilisé le mot-clé synchronized dans la méthode incrementerCompteur().
À chaque opération calculée, le compteur est incrémenté et sa nouvelle valeur est affichée dans la console du serveur.

8. Adaptation du client

La classe ClientObjet reprend le fonctionnement du client développé dans le TP2.
Elle permet à l'utilisateur de saisir deux nombres et un opérateur mathématique. Ces informations sont ensuite regroupées dans un objet Operation.
L'objet est transmis au serveur grâce à ObjectOutputStream et à la méthode writeObject().
Enfin, le client reçoit le résultat du calcul à l'aide d'un flux de lecture et l'affiche à l'écran.
Le serveur retourne le résultat sous forme de texte à travers PrintWriter, tandis que le client le récupère à l'aide de BufferedReader.

ClientObjet.java

```java
package act2.client;
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.io.ObjectOutputStream;
import java.io.IOException;
import java.net.Socket;
import java.util.Scanner;
import act2.op.Operation;

public class ClientObjet {

    public static void main(String[] args) {

        try (
                Socket socket = new Socket( host: "localhost", port: 1234)) {
            System.out.println("Connecte au serveur.");
            // Creation des flux
            ObjectOutputStream oos = new ObjectOutputStream(socket.getOutputStream());
            BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
            oos.flush();
            // Saisie utilisateur
            Scanner sc = new Scanner(System.in);
            System.out.print("Entrez le premier nombre : ");
            double a = sc.nextDouble();
            System.out.print("Entrez l'operateur (+, -, *, /) : ");
            String operateur = sc.next();
            System.out.print("Entrez le deuxieme nombre : ");
            double b = sc.nextDouble();
            // Creation et envoi de l'objet
            Operation operation = new Operation(a, operateur, b);
            oos.writeObject(operation);
            oos.flush();
            // Reception du resultat
            String resultat = in.readLine();
            System.out.println("Resultat recu du serveur : " + resultat);

        } catch (IOException e) {
            System.out.println("Erreur de connexion : " + e.getMessage());
        }
    }
}
```

Figure 6 – Code de la classe ClientObjet.java

Première vérification de la communication client-serveur

L'application repose sur une architecture client-serveur utilisant le protocole TCP et l'échange d'objets Java.
Au démarrage, le serveur se met en attente des connexions entrantes sur le port 1234.
Lorsqu'un client établit une connexion, le serveur accepte celle-ci et crée un nouveau thread chargé de traiter les échanges avec ce client.

Du côté client, l'utilisateur saisit deux nombres ainsi qu'un opérateur mathématique parmi les quatre opérations disponibles : addition, soustraction, multiplication et division. Ces informations sont regroupées dans un objet de la classe Operation, puis envoyées au serveur à l'aide de ObjectOutputStream.

Le serveur reçoit l'objet avec ObjectInputStream, récupère les données nécessaires et effectue le calcul correspondant.
Après chaque opération calculée, le compteur global est incrémenté à travers une méthode synchronisée. La nouvelle valeur du compteur est ensuite affichée dans la console du serveur.
Enfin, le résultat est transmis au client, qui le reçoit et l'affiche à l'écran.

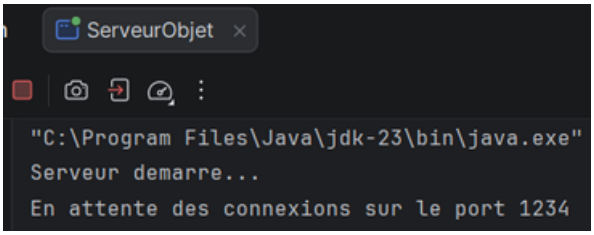


Figure 7 – Console du serveur démarré

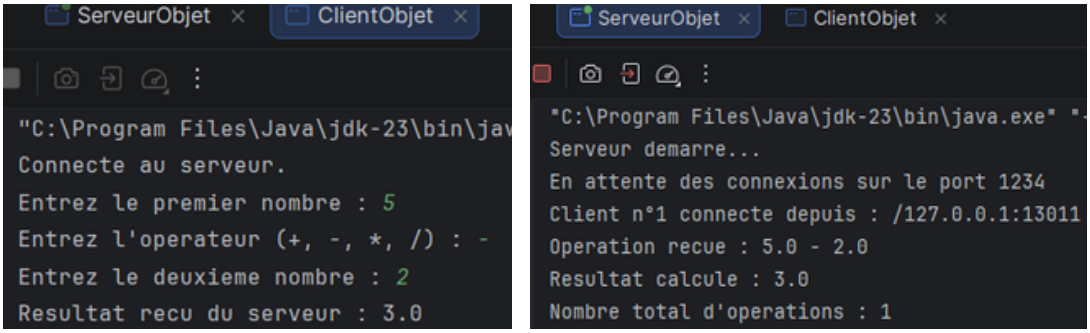


Figure 8 – Console serveur et console client après la connexion

Tester plusieurs clients

Configuration d'IntelliJ IDEA pour l'exécution simultanée de plusieurs clients:

Afin de tester le fonctionnement multithread de notre application, nous avons configuré IntelliJ IDEA pour autoriser l'exécution de plusieurs instances du programme client ClientObjet.
Cette configuration permet de lancer plusieurs clients indépendamment, tout en maintenant le serveur actif. Elle facilite ainsi la simulation de plusieurs connexions simultanées au serveur.

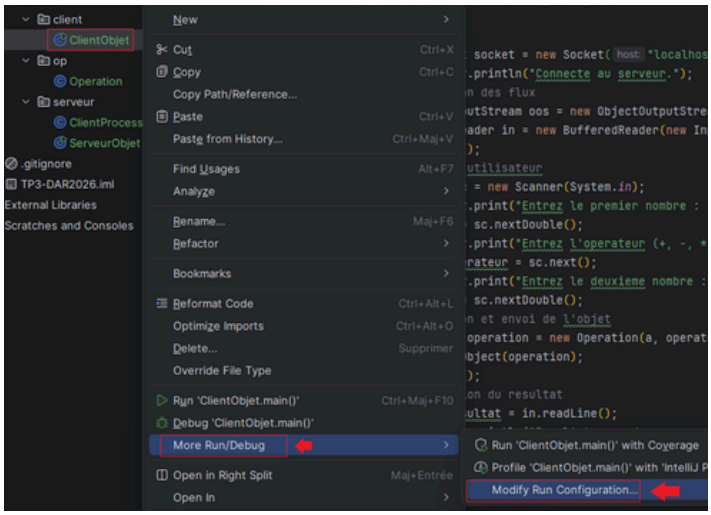


Figure 9 – Accès à la configuration du client

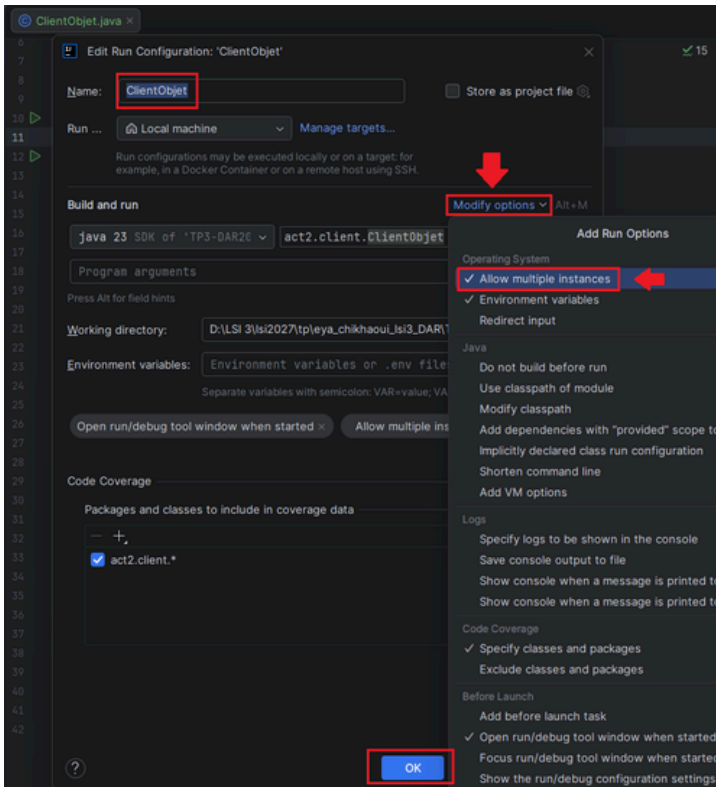


Figure 10 – Activation de Allow multiple instances

Afin de vérifier le fonctionnement multithread du serveur, nous avons lancé plusieurs clients simultanément. Chaque connexion est acceptée par le serveur et prise en charge par un thread indépendant grâce à la classe ClientProcess. Chaque client peut ainsi envoyer une opération mathématique et recevoir son propre résultat, tandis que le serveur continue à accepter de nouvelles connexions. Ce test permet également de vérifier le partage du compteur global entre les différents threads et son incrémentation après chaque opération calculée.

Analyse des résultats :
Le test réalisé avec plusieurs clients permet de vérifier le fonctionnement multithread du serveur. Comme le montre la capture, deux clients ont réussi à se connecter au serveur simultanément, chacun disposant de sa propre connexion et de son propre thread. Le premier client a envoyé l'opération de division 5 / 2, le serveur a effectué le calcul et obtenu le résultat 2.5, qui a été correctement transmis et affiché dans la console du client. Ensuite, le deuxième client a envoyé l'opération d'addition 7 + 8, donnant un résultat de 15.0, également reçu correctement par le client correspondant. On remarque que le compteur global des opérations passe de 1 à 2 après le traitement des deux requêtes. Ce test confirme ainsi que le serveur est capable de gérer plusieurs clients à travers des threads indépendants, de traiter leurs opérations et de leur transmettre les résultats correspondants.

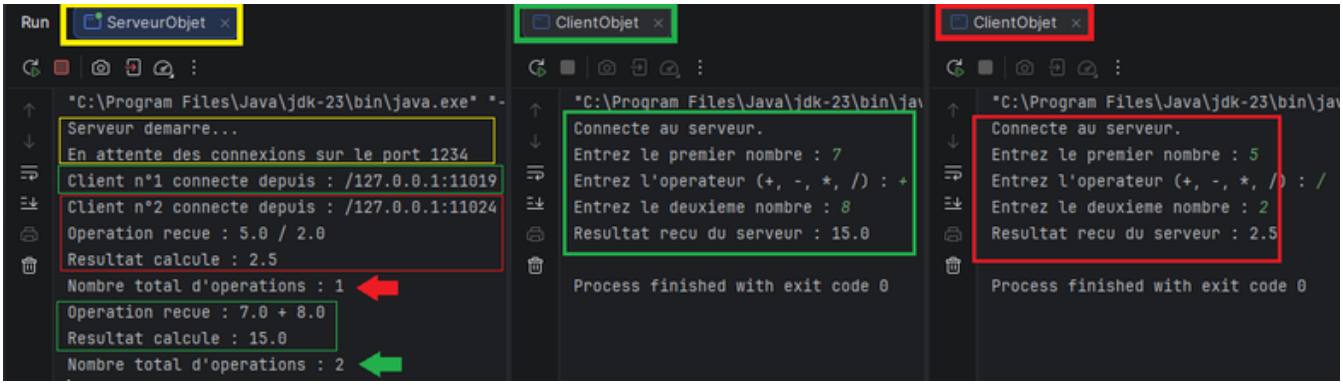


Figure 11 – Test multithread : connexion de deux clients et traitement d'une opération mathématique par le serveur.

lien de tp dans github:
https://github.com/eyaChikhaoui/TP3-DAR2026.git

Conclusion :
À travers cette activité, nous avons amélioré l'application calculatrice en combinant la communication par échange d'objets Java et le traitement multithread.
Nous avons mis en pratique la sérialisation des objets avec Serializable, ObjectOutputStream et ObjectInputStream, ainsi que la création de threads pour permettre au serveur de traiter plusieurs clients indépendamment.
Nous avons également introduit un compteur global partagé entre les différents threads et protégé son incrémentation grâce à la synchronisation avec synchronized.
Cette activité nous a permis de mieux comprendre la communication entre plusieurs clients et un serveur, ainsi que l'importance de la synchronisation dans une application répartie.

Je reste disponible pour toute remarque ou suggestion d'amélioration.
Merci pour votre attention.